

Memoria del Proyecto Final

Dashboard de Trading de Criptoactivos

Introducción a Python

Rafael Ayudarte López
rayudarte@hesperides.edu.es

Enero 2026

Abstract

Este proyecto implementa un dashboard interactivo para visualizar y analizar estrategias de trading de criptoactivos. El sistema combina un backend en FastAPI con arquitectura hexagonal, un frontend en React, e integración con Binance mediante REST API y WebSockets. Persistencia en PostgreSQL con idempotencia mediante UPSERT, estrategias cuantitativas (cruce de medias, momentum, patrones de velas), y métricas de rendimiento del portafolio.

Contents

1	Introducción	4
1.1	Contexto del Proyecto	4
1.2	Tecnologías Seleccionadas	4
1.3	Justificación de Elecciones	4
1.4	Activos Seleccionados	4
2	Ordenación Temporal, Latencia y Relojes Distribuidos	4
2.1	Preguntas Básicas [1 pt]	4
2.2	Preguntas Avanzadas [1 pt]	5
3	Persistencia, Idempotencia y Eliminación de Duplicados	6
3.1	Preguntas Básicas [1 pt]	6
3.2	Preguntas Avanzadas [1 pt]	7
4	Backpressure y Control de Flujo	8
4.1	Preguntas Básicas [1 pt]	8
4.2	Preguntas Avanzadas [1 pt]	8
5	Consistencia Eventual vs Consistencia Fuerte en PnL	9
5.1	Preguntas Básicas [1 pt]	9
5.2	Preguntas Avanzadas [1 pt]	9
6	Modelado Avanzado de Posiciones y Agregados	10
6.1	Preguntas Básicas [1 pt]	10
6.2	Preguntas Avanzadas [1 pt]	12
7	Concurrencia, Asincronismo y Paralelismo en Python	12
7.1	Preguntas Básicas [1 pt]	12
7.2	Preguntas Avanzadas [1 pt]	13
8	Uso Avanzado de Python: Decoradores, Metaclases y Dunder Methods	15
8.1	Preguntas Básicas [1 pt]	15
8.2	Preguntas Avanzadas [1 pt]	18
9	Escalado Horizontal y Multiactivo	21
9.1	Preguntas Básicas [1 pt]	21
9.2	Preguntas Avanzadas [1 pt]	21
10	Tolerancia a Fallos y Recuperación Determinista	22
10.1	Preguntas Básicas [1 pt]	22
11	Pruebas Avanzadas y Validación del Sistema	23
11.1	Preguntas Básicas [1 pt]	23
11.2	Preguntas Avanzadas [1 pt]	26
12	Arquitectura del Sistema	27
12.1	Arquitectura Hexagonal	27

13 Conclusiones	27
13.1 Retos Encontrados	27
13.2 Decisiones de Simplificación	28
13.3 Posibles Mejoras Futuras	28
13.4 Comentarios	28

1 Introducción

1.1 Contexto del Proyecto

Este proyecto forma parte del proceso de selección para la posición de Analista Cuantitativo Junior en Goldman Stanley. El objetivo es desarrollar un sistema completo de trading de criptoactivos que demuestre competencias en Python, arquitectura de software, procesamiento de datos en tiempo real y desarrollo full-stack. Se ha desarrollado un Dashboard interactivo que te permite ver criptoactivos, en tiempo real te permite crear un motor de trading que siga distintas estrategias long only que se ejecutan `ON_CLOSE`. También está implementado el Backtest de las estrategias.

1.2 Tecnologías Seleccionadas

- **Backend:** FastAPI con arquitectura hexagonal (Domain-Driven Design)
- **Frontend:** React con Vite
- **Base de datos:** PostgreSQL con asyncpg
- **Otros:** asyncio para concurrencia, aiohttp para HTTP/WebSocket, tenacity para reintentos, Docker para containerización

1.3 Justificación de Elecciones

- **FastAPI:** Framework asíncrono moderno con tipado estático, documentación automática OpenAPI, y excelente rendimiento para operaciones I/O-bound como WebSockets.
- **PostgreSQL:** Base de datos relacional robusta con soporte nativo para transacciones ACID, índices eficientes, y operaciones UPSERT para idempotencia.
- **asyncpg:** Driver PostgreSQL asíncrono 2-3x más rápido que psycopg2, compatible con el event loop de asyncio.
- **Arquitectura hexagonal:** Separación clara entre dominio, aplicación e infraestructura, facilitando testing y mantenibilidad.

1.4 Activos Seleccionados

- BTC, ETH, XRP, BNB, SOL, TRX, DOGE, ADA, LINK, HYPE (10 principales)
- Sistema escalado a 150 símbolos para pruebas de carga

2 Ordenación Temporal, Latencia y Relojes Distribuidos

2.1 Preguntas Básicas [1 pt]

Pregunta Básica

¿Cómo has detectado que los WebSockets de Binance pueden entregar mensajes fuera de orden?

No he conseguido simular el escenario en el que los WebSockets de Binance pueden entregar mensajes fuera de orden. Pero en el caso de que ocurriera, usaría un Buffer para reordenar los mensajes y asegurar que la secuencia de timestamps sea monótona.

Pregunta Básica

¿Qué diferencia hay entre event time e ingestion time? ¿Cuál usarías para cálculos y por qué?

El event time es el tiempo real del evento, mientras que el ingestion time es el tiempo en que el evento es ingerido por el sistema. Para realizar cálculos usaría el event time ya que es el tiempo real del evento y queremos que nuestros cálculos sean lo más precisos posibles.

En mi implementación, uso `open_time` (event time de Binance) para todos los cálculos de estrategias y ordenación de velas.

Pregunta Básica

¿Cómo garantizas en Python que una secuencia de timestamps cumpla monotonicidad temporal?

Garantizo monotonicidad mediante dos mecanismos:

1. Para sacar datos de la REST API y meterlos en la BBDD no tengo que preocuparme por la monotonía porque luego en la base de datos puedo filtrar por `open_time DESC`.
2. Pero en el caso de que ocurriera, usaría un Buffer para reordenar los mensajes y asegurar que la secuencia de timestamps sea monótona.

```
1 # En PostgresAdapter - orden garantizado por SQL
2 async def get_candles(self, symbol: Symbol, interval: TimeFrame, limit:
   int):
3     return await self._pool.fetch("""
4         SELECT * FROM candles
5         WHERE symbol = $1 AND interval = $2
6         ORDER BY open_time DESC -- Monotonicidad garantizada
7         LIMIT $3
8     """, str(symbol), interval.value, limit)
```

2.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Describe cómo reconstruyes un flujo temporal coherente cuando llegan ticks fuera de orden.

Implementé un `CandleBuffer` que actúa como buffer de reordenación con ventana temporal. Cuando llegan velas fuera de orden:

1. Las velas se insertan en el buffer ordenadas por `open_time`, cuando esta cierra y se hacen varias comprobaciones para garantizar monotonía.

Pregunta Avanzada

Define clock drift. ¿En qué casos el timestamp del backend es autoritativo sobre el del exchange?

Clock drift es la diferencia progresiva entre dos relojes que no están sincronizados. En sistemas distribuidos, el reloj del backend puede desviarse del reloj de Binance.

El timestamp del backend es autoritativo cuando:

- Se registra el momento de recepción para cálculos de latencia de red
- Se necesita auditoría del sistema (cuándo llegó realmente el dato)

El timestamp del exchange es autoritativo cuando:

- Se calculan estrategias de trading (el precio “ocurrió” en ese momento)
- Se ordenan velas cronológicamente
- Se detectan gaps en datos históricos

3 Persistencia, Idempotencia y Eliminación de Duplicados

3.1 Preguntas Básicas [1 pt]

Pregunta Básica

¿Cómo identificas duplicados tras una reconexión al WebSocket? ¿Cuál es la primary identity de un trade?

Identifico duplicados mediante la **primary identity** compuesta por (**symbol**, **interval**, **open_time**) para velas. Esta combinación es única ya que solo puede existir una vela de BTC/USDT en timeframe 1h que abra a las 14:00 del 1 de enero. Además usando un Buffer la vela del websocket solo se introduce al buffer una vez cerrada donde se hacen las comprobaciones de monotonía, si hay alguna violación de monotonía en el buffer se indica y se recupera la salud de este.

Para trades, la primary identity sería (**trade_id**, **symbol**) donde **trade_id** es el identificador único asignado por Binance. Al guardar trades en la base de datos le asigno un **trader_id** usando uuid.

Pregunta Básica

Define idempotencia en el contexto de tu ingestor de datos.

La idempotencia significa que ejecutar la misma operación múltiples veces produce el mismo resultado que ejecutarla una sola vez.

En mi ingestor es esencial la idempotencia para evitar registrar dos veces la misma vela.

Pregunta Básica

Describe tu mecanismo para evitar registrar dos veces el mismo trade.

Para no registrar dos veces el mismo trade, uso un **trader_id** que es único para cada trader, utilizamos la PRIMARY KEY (**trade_id**), la base de datos que nos garantiza idempotencia. De

forma similar, para las velas tomamos una restricción extra con UNIQUE (symbol, interval, open_time) para evitar duplicados. Utilizando UPSERT (INSERT ... ON CONFLICT DO NOTHING) en PostgreSQL:

```

1  # modules/trading/infrastructure/postgre_adapter.py
2  async def save_candle(self, candle: Candle_static) -> None:
3      async with self._pool.acquire() as conn:
4          await conn.execute("""
5              INSERT INTO candles (symbol, interval, open_time,
6                                  open, high, low, close, volume)
7              VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
8              ON CONFLICT (symbol, interval, open_time) DO NOTHING
9          """,
10             str(candle.symbol),
11             candle.interval.value,
12             candle.timestamp.timestamp,
13             candle.open.value,
14             candle.high.value,
15             candle.low.value,
16             candle.close.value,
17             candle.volume.value
18         )

```

El ON CONFLICT DO NOTHING ignora silenciosamente intentos de insertar duplicados que son rechazados por UNIQUE, haciendo la operación idempotente.

3.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Si usas SQL, detalla tu esquema mínimo para garantizar unicidad bajo concurrencia.

La concurrencia en SQL se consigue a través de las primary keys con UNIQUE CONSTRAINT.

```

1  -- Schema para candles con unicidad garantizada
2  CREATE TABLE IF NOT EXISTS candles (
3      id SERIAL PRIMARY KEY,
4      symbol VARCHAR(20) NOT NULL,
5      interval VARCHAR(10) NOT NULL,
6      open_time TIMESTAMP NOT NULL,
7      open DECIMAL(20, 8) NOT NULL,
8      high DECIMAL(20, 8) NOT NULL,
9      low DECIMAL(20, 8) NOT NULL,
10     close DECIMAL(20, 8) NOT NULL,
11     volume DECIMAL(20, 8) NOT NULL,
12     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
13
14     -- CRITICAL: Constraint de unicidad para UPSERT
15     CONSTRAINT unique_candle UNIQUE (symbol, interval, open_time)
16 );
17
18 -- Índices para queries rápidas
19 CREATE INDEX idx_candles_symbol_interval ON candles(symbol, interval);
20 CREATE INDEX idx_candles_open_time ON candles(open_time DESC);

```

4 Backpressure y Control de Flujo

4.1 Preguntas Básicas [1 pt]

Pregunta Básica

Define qué es backpressure y dónde aparece en tu sistema.

Backpressure es el mecanismo por el cual un consumidor lento señala al productor que debe reducir su velocidad de envío. En mi sistema podría aparecer en el motor de trading en las diferentes **Queues** que hay en el motor de trading. Si el WS manda más rápido las velas de lo que tardan en recogerse de la cola esto puede generar backpressure. De hecho en el motor hay más colas por lo que si alguna parte del proceso, ya sea el análisis de señales, la ejecución de la orden con el exchange o la actualización del portfolio tarda más de lo normal puede generar backpressure en otras colas.

Pregunta Básica

¿Qué ocurre si Binance envía datos más rápido de lo que procesas?

En mi caso no se llena nunca la cola ya que estoy conectado al stream de las klines por lo que me manda un tick cada dos segundos que es tiempo de sobra para procesar la vela.

Pero si Binance manda datos más rápido de lo que puedes procesar, puede generar que las señales que se den sean incorrectas y por lo tanto hacer trades erróneos.

Pregunta Básica

¿Cómo detectas en Python que tu consumidor va con retraso (lag)?

Podemos observar si la cola donde se están depositando las velas se está llenando. Además cronometrar con un decorador cuánto tarda el consumidor en procesar la vela y emitir la información correspondiente.

4.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Compara bounded queues con drop oldest y rate limiting. ¿Qué sesgos introduce cada una?

1. **Bounded Queue con drop oldest:** Elimina los datos más antiguos cuando la cola está llena, priorizando información reciente. Introduce sesgo de recencia - pierde contexto histórico y patrones largos, creando visión miope del mercado. En trading, esto puede hacer perder tendencias importantes pero garantiza procesar datos actuales.
2. **Rate Limiting:** Limita la tasa de ingesta, rechazando datos que exceden un umbral por segundo. Introduce sesgo de volatilidad - subestima picos de mercado y eventos de alta frecuencia. Filtra ruido pero pierde momentos críticos donde la velocidad de datos contiene información valiosa para decisiones de trading.

En mi caso para el candlebuffer uso **Deque** que elimina la vela más antigua. Sin embargo cuando estoy recibiendo ticks del WebSocket uso **asyncio.Queue** con tamaño máximo, que blo-

quearía el proceso si se llena la cola. Esto es útil porque si voy un número grande de velas atrasadas, no quiero que siga operando.

Pregunta Avanzada

Describe cómo reequilibrarías el throughput del pipeline bajo carga extrema.

Estrategias de reequilibrio:

1. **Reducir símbolos:** Desuscribirse de símbolos menos prioritarios temporalmente
2. **Aumentar batch size:** Agrupar más velas por escritura a PostgreSQL
3. **Escalar workers:** Usar `multiprocessing` para cálculos CPU-bound
4. **Priorizar:** Procesar primero activos con posiciones abiertas

5 Consistencia Eventual vs Consistencia Fuerte en PnL

5.1 Preguntas Básicas [1 pt]

Pregunta Básica

Indica qué partes necesitan consistencia fuerte para calcular correctamente el PnL.

La consistencia fuerte es una garantía de que todos los nodos de un sistema distribuido ven los mismos datos al mismo tiempo, en el mismo orden, y justo después de que una operación se completa.

- **PnL de cada trader:** Cantidad y precio medio de entrada deben ser exactos
- **Historial de trades:** Cada trade cerrado para calcular PnL realizado

Tengo una entidad cuando se inicia la aplicación que se encarga de gestionar los estados de trading, para que cada portfolio corra en el background y para acceder a estos se usan Locks. Soy consciente que no uso Locks para garantizar la consistencia al hacer cambios en la base de datos, pretendo cambiarlo en el futuro.

Pregunta Básica

Da un ejemplo donde un retraso o inconsistencia genera un PnL incorrecto.

Puede ocurrir en el portfolio donde operan los distintos traders. Si uno quiere cambiar la PnL del portfolio y llega otro y la cambia cuando el otro aún no había acabado de actualizar el PnL, esto generará inconsistencias. En mi caso puede ocurrir que quiera leer la PnL del portfolio y que se ejecute alguna compra o venta a la mitad y me devolviera un PnL incorrecto.

5.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Describe condiciones de carrera en cálculos de PnL y cómo las has eliminado.

El problema descrito antes es una condición de carrera. Lo que tendría que hacer es incluir Locks cuando estoy haciendo modificaciones sobre la clase Portfolio (esto no está implementado).

Pregunta Avanzada

Explica qué partes del sistema pueden ser eventualmente consistentes sin afectar decisiones.

Partes que toleran consistencia eventual, estas partes no son críticas para la toma de decisiones, con que se actualicen correctamente en el futuro podemos estar tranquilos:

- Trades realizados
- Estadísticas agregadas
- Velas cerradas de la base de datos

6 Modelado Avanzado de Posiciones y Agregados

6.1 Preguntas Básicas [1 pt]

Pregunta Básica

Define la entidad responsable de mantener la posición agregada. Explica su invariante.

La entidad portfolio se encarga de mantener la posición agregada de distintos traders. Se le asigna un capital inicial que reparte entre los traders y verifica que haya un número positivo de traders.

```

1  def __post_init__(self) -> None:
2      # Invariant 1: initial capital must be positive
3      if self.initial_capital <= 0:
4          raise ValueError(f"initial_capital must be > 0, got: {self.
5                               initial_capital}")
6
7      # Invariant 2: number of traders must be positive
8      _num_traders = len(self.traders)
9      if _num_traders <= 0:
10         raise ValueError(f"number of traders must be > 0, got: {
11                               _num_traders}")
12
13     self._capital_for_trader = self.initial_capital / _num_traders
14
15     self._current_capital = {trader.id: PnL(self._capital_for_trader)
16                               for trader in self.traders}
17
18     self._trades = {trader.id: [] for trader in self.traders}
19
20     logger.info(f"Portfolio created with capital: {self.initial_capital}
21                 and {_num_traders} traders")

```

No tengo una entidad que se encargue de la posición agregada de los distintos portfolios. Solamente tengo una función en el controller de Portfolio que saca información de los distintos portfolios iterando sobre ellos lo que podría generar condiciones de carrera.

Pregunta Básica

Explica con un ejemplo cómo actualizas el precio medio tras varios partial fills.

Aplico estrategias long only en la que cada trader se encarga de lanzar las señales. En el caso que no tenga una posición abierta y tenga señal de compra, comprará todo el activo que pueda con el dinero que tiene asignado a precio de mercado. Y solo se venderá y cerrará la posición siempre que haya una señal de venta con una posición abierta y se vende todo el activo al precio de mercado.

A la hora de crear la orden y mandarla a Binance especifico que compre a mercado. Por lo que no tengo que preocuparme por los partial fills.

```

1
2 @timed_async
3 async def buy_market(self, symbol: Symbol, quantity: Quantity) ->
  OrderResponse:
4     """Buy at current market price."""
5     if not self._client:
6         raise RuntimeError("Not connected. Call connect() first.")
7
8     try:
9         response = await self._client.create_order(
10             symbol=str(symbol),
11             side="BUY",
12             type=ORDER_TYPE_MARKET,
13             quantity=str(quantity.value)
14         )
15
16         return self._parse_response(response, symbol, Side.BUY)
17
18     except Exception as e:
19         logger.error(f"Error buying market: {e}")
20         raise
21
22 @timed_async
23 async def sell_market(self, symbol: Symbol, quantity: Quantity) ->
  OrderResponse:
24     """Sell at current market price."""
25     if not self._client:
26         raise RuntimeError("Not connected. Call connect() first.")
27
28     try:
29         response = await self._client.create_order(
30             symbol=str(symbol),
31             side="SELL",
32             type=ORDER_TYPE_MARKET,
33             quantity=str(quantity.value)
34         )
35
36         return self._parse_response(response, symbol, Side.SELL)
37
38     except Exception as e:
39         logger.error(f"Error selling market: {e}")
40         raise

```

6.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Explica problemas si tus entidades mezclan conceptos específicos de Binance con el dominio.

Si el dominio mezcla lógica que funciona con las características que ofrece Binance puede generar problemas cuando quiera implementar otros exchanges. El dominio debe ser totalmente agnóstico a lo que ofrece Binance o cualquier otro exchange.

Por ejemplo, si mi entidad `Candle` tiene un campo `binance_trade_count` específico de Binance, cuando implemente un adaptador para Kraken (que no proporciona ese dato), tendré que:

- Hacer el campo opcional (contamina el dominio)
- Inventar un valor (datos incorrectos)
- Refactorizar el dominio (costoso)

La solución correcta es que el dominio defina solo lo esencial (`OHLCV`) y los detalles específicos queden en el adaptador.

7 Concurrencia, Asincronismo y Paralelismo en Python

7.1 Preguntas Básicas [1 pt]

Pregunta Básica

Explica cuándo usar `asyncio` y cuándo usar `threads`.

Teóricamente: Uso `asyncio` para operaciones I/O bound como conexiones a APIs, bases de datos y WebSockets, donde el tiempo de espera domina. Su modelo de event loop maneja miles de conexiones concurrentes en un solo hilo, ideal para sistemas de trading que consumen datos de mercado en tiempo real. Uso `threads` para operaciones CPU bound como cálculos intensivos, procesamiento de datos complejos o algoritmos pesados que pueden paralelizarse. Cada thread tiene su propio GIL pero aprovecha múltiples núcleos para tareas computacionalmente costosas que bloquean la ejecución.

En mi proyecto uso **`asyncio`** para toda la ingesta (REST, WebSocket, PostgreSQL) porque son operaciones I/O-bound donde el 95% del tiempo es espera de red. No he implementado `threads` en ninguna parte del proyecto pero se me ocurre que cuando corro la aplicación de FastAPI podría tener un hilo que se encargase de tener la posición agregada de los distintos portafolios. También podría tener un hilo que se encargase de hacer el backtest de las estrategias ya que ahora mismo me bloquea el event loop durante unos segundos.

Pregunta Básica

Define qué problema resuelve el GIL. ¿Cómo afecta al procesamiento en tu sistema?

El **Global Interpreter Lock (GIL)** es un mutex que protege el acceso a objetos Python, garantizando que solo un thread ejecuta bytecode Python a la vez. Resuelve el problema de race conditions en estructuras internas de CPython.

Aunque no tenga implementado ningún hilo si lo tuviera en el backtest no podría obtener paralelismo real con el resto de la aplicación porque solo se ejecuta un hilo a la vez.

Para tareas CPU-bound usaría `multiprocessing`. De hecho lo he usado para un backtest de estrategias de cruce de medias. Pensando sobre cómo funciona la estrategia se me hacía raro que comprases cuando ya llevaba subiendo un rato (la media corta cruza la larga) y que vendieses cuando ya llevaba bajando un rato (la media corta cruza la larga). Si vendes cuando lleva bajando un rato prácticamente nunca ganas dinero, quería comprobar si existía un porcentaje de take profit óptimo que mande señal de venta si se ha conseguido cierta rentabilidad. Utilizando el servicio de backtest que tenía hecho y `multiprocessing` ejecuté la estrategia para un rango de porcentajes de take profits para cada intervalo y para diferentes combinaciones de medias cortas y largas. Al final eran más de 200 casos, pero era la misma función en todos ellos por eso usé `multiprocessing`.

Pregunta Básica

¿Cómo asegurarías que una función bloqueante no congela tu event loop?

Tengo un evento que me bloquea el event loop, el backtest bloquea el event loop durante los segundos que tarda en completarse. Esto lo solucionaría con `ProcessPoolExecutor`, liberando el event loop principal para seguir atendiendo requests. Con `loop.run_in_executor()`, el servidor mantiene responsividad mientras el backtest procesa en paralelo, evitando que otros usuarios queden bloqueados.

7.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Diseña un pipeline concurrente para ingesta, almacenamiento y cálculo usando `asyncio.Queue`.

He diseñado un motor de trading que primero carga las velas necesarias históricas a los traders antes de empezar a operar. Una vez empieza recibe velas del WebSocket y las mete en una cola. Los traders van cogiendo las velas que utilizan y procesan señales y si esas señales cumplen los requisitos crean órdenes, las ponen en otra cola para que sean ejecutadas. Una vez ejecutadas las órdenes la respuesta que se recibe de Binance se pone en otra cola y se actualiza el portfolio. Esto es posible ya que creo las Tasks y ejecuto las corrutinas a la vez con `TaskGroup`.

```
1  async def startup(self) -> None:
2      """Initialize trading engine: restore portfolio state and warm-up
3         traders."""
4      if self._portfolio_manager is None:
5          raise ValueError("Portfolio manager must be initialized before
6             calling startup()")
7
8      # Step 1: Restore portfolio state from database (or create new)
9      await self._portfolio_manager.restore_state()
10
11     # Step 2: Warm-up traders with historical data
12     await self._warm_up_traders()
13
14 async def run(self) -> None:
15     """Run the trading pipeline."""
16     self._running = True
```

```

15
16     try:
17         async with TaskGroup() as tg:
18
19             for (symbol, interval) in self._traders_by_pair.keys():
20                 tg.create_task(self._stream_task(symbol, interval))
21
22
23             tg.create_task(self._signal_processor())
24
25
26             tg.create_task(self._order_executor())
27
28
29             tg.create_task(self._portfolio_updater())
30
31     except* Exception as e:
32         logger.error(f"Trading engine error: {e}")
33         raise

```

Pregunta Avanzada

Explica una situación donde await puede introducir un race condition y cómo evitarlo.

Por ejemplo, si tienes algún await cuando estás modificando un recurso compartido, estás liberando el event loop causando que otra corrutina pueda venir a por el recurso compartido pero aún no te ha dado tiempo de modificarlo.

En mi caso necesito saber cuántos portfolios están operando a la vez y para ello uso un Lock() cuando accedo a ellos con corrutinas para evitar que dos corrutinas accedan a ellos a la vez.

Race condition real: Cuando queremos actualizar la posición de un portfolio y dos traders van a cambiar el PnL a la vez.

```

1  # PROBLEMA: Race condition con await
2  async def update_portfolio(trader_id, pnl_delta):
3      current = await db.get_pnl(portfolio_id) # await 1
4      # <-- Otro task puede modificar aqui!
5      new_pnl = current + pnl_delta
6      await db.set_pnl(portfolio_id, new_pnl) # await 2
7
8  # SOLUCION: Lock asyncio
9  portfolio_lock = asyncio.Lock()
10
11  async def update_portfolio_safe(trader_id, pnl_delta):
12      async with portfolio_lock:
13          current = await db.get_pnl(portfolio_id)
14          new_pnl = current + pnl_delta
15          await db.set_pnl(portfolio_id, new_pnl)

```

8 Uso Avanzado de Python: Decoradores, Metaclases y Dunder Methods

8.1 Preguntas Básicas [1 pt]

Pregunta Básica

¿Qué responsabilidad cumple el decorador personalizado que exigía la práctica?

He implementado dos decoradores:

1. **@timed**: Mide el tiempo que tarda una función síncrona en ejecutarse.
2. **@timed_async**: Mide el tiempo que tarda una función asíncrona en ejecutarse.

Sirven para medir el tiempo que tarda una función en ejecutarse, devuelve un log en debug con el nombre de la función y el tiempo que ha tardado en ejecutarse. Es útil cuando crees que hay lag en tu sistema, se verá la implementación más adelante.

Pregunta Básica

Explica qué ventaja aporta una metaclass en tu diseño concreto.

La metaclass `AdapterMeta` valida automáticamente que los adaptadores implementan correctamente los puertos hexagonales. La ventaja principal es que detecta errores en tiempo de definición de clase, no en tiempo de ejecución, lo que permite identificar problemas de implementación antes de que el código se ejecute.

```
1 class AdapterMeta(ABCMeta):
2     """
3     Metaclass that checks that an adapter implements all the methods
4     defined in its port (interface).
5     """
6
7     def __new__(mcs, name: str, bases: tuple, namespace: Dict[str, Any])
8         -> Type:
9         cls = super().__new__(mcs, name, bases, namespace)
10
11         # Auto-detect port from base classes (find ABC subclass that
12         # ends with "Port")
13         port = None
14         for base in bases:
15             if base.__name__.endswith('Port') and issubclass(base, ABC):
16                 port = base
17                 break
18
19         if port is None:
20             return cls
21
22         # Get the abstract methods of the port
23         port_methods = set()
24         for method_name in dir(port):
25             if method_name.startswith('_'):
26                 continue
```

```

25         method = getattr(port, method_name)
26         if callable(method) and getattr(method, '
           __isabstractmethod__', False):
27             port_methods.add(method_name)
28
29         # Check that all are implemented
30         missing = []
31         wrong_signature = []
32
33         for method_name in port_methods:
34             if not hasattr(cls, method_name):
35                 missing.append(method_name)
36                 continue
37
38             adapter_method = getattr(cls, method_name)
39
40             # Check that it is not still abstract
41             if getattr(adapter_method, '__isabstractmethod__', False):
42                 missing.append(method_name)
43                 continue
44
45             # Check the signature (parameters)
46             port_method = getattr(port, method_name)
47             port_sig = inspect.signature(port_method)
48             adapter_sig = inspect.signature(adapter_method)
49
50             port_params = list(port_sig.parameters.keys())
51             adapter_params = list(adapter_sig.parameters.keys())
52
53             if port_params != adapter_params:
54                 wrong_signature.append(
55                     f"{method_name}: esperado {port_params}, tiene {
                       adapter_params}"
56                 )
57
58         # Report errors
59         errors = []
60
61         if missing:
62             errors.append(f"Methods not implemented: {missing}")
63
64         if wrong_signature:
65             errors.append(f"Incorrect signatures: {wrong_signature}")
66
67         if errors:
68             raise TypeError(
69                 f"Adapter '{name}' does not comply with '{port.__name__
                       }':\n    " +
70                 "\n    ".join(errors)
71             )
72
73         logger.debug(f"{name} implements {port.__name__} correctly")
74         return cls

```

Ventajas:

- **Detección temprana:** Los errores se detectan al definir la clase, no al ejecutarla.
- **Detección automática del puerto:** Busca automáticamente clases base que terminen en "Port" y sean ABC.
- **Validación de firmas:** Verifica que los parámetros de los métodos coincidan con el puerto.
- **Mensajes de error claros:** Indica qué métodos faltan y qué firmas son incorrectas.

Pregunta Básica

Menciona dos dunder methods usados y explica cómo mejoran la semántica de tus objetos.

He utilizado extensivamente dos tipos de dunder methods en mi sistema:

1. `__str__`: Implementado en `Candle_static` para poder entender los objetos de forma visual y clara. Mejora la semántica permitiendo que al imprimir una vela se muestre toda la información relevante de forma estructurada, incluyendo indicadores visuales (verde/rojo) y cálculos derivados como el cambio porcentual.
2. `__aenter__` y `__aexit__`: Implementados en `TradingEngine` como context manager asíncrono. Mejoran la semántica permitiendo usar `async with` para gestionar el ciclo de vida del motor de trading, asegurando que todas las conexiones se abran y cierren correctamente, lo que facilita enormemente el testing.

```

1  @dataclass(frozen=True, slots=True)
2  class Candle_static:
3      """OHLCV data of a candle."""
4      symbol: Symbol
5      open: Price
6      high: Price
7      low: Price
8      close: Price
9      volume: Quantity
10     interval: Interval
11     timestamp: Timestamp
12
13     def __str__(self) -> str:
14         direction = "UP" if self.close.value >= self.open.value else "
15             DOWN"
16         change = self.close.value - self.open.value
17         change_pct = (change / self.open.value) * 100
18
19         return (
20             f"{direction} {self.symbol} [{self.interval.value}]\n"
21             f"  Time:    {self.timestamp.timestamp:%Y-%m-%d %H:%M:%S}\n"
22             f"  Open:    ${self.open.value:>10,.2f}\n"
23             f"  High:    ${self.high.value:>10,.2f}\n"
24             f"  Low:     ${self.low.value:>10,.2f}\n"
25             f"  Close:   ${self.close.value:>10,.2f}\n"
26             f"  Change:  ${change:>10,.2f} ({change_pct:+6.2f}%)\n"
27             f"  Volume:  {self.volume.value:>10,.4f}"
28         )

```

```

29     def __repr__(self) -> str:
30         return f"<Candle {self.symbol} {self.interval.value} @ {self.
            close.value}>"

```

El `__str__` permite que al hacer `print(candle)` se muestre toda la información de forma legible, facilitando el debugging y la comprensión del estado de las velas durante el desarrollo.

```

1 class TradingEngine:
2     async def __aenter__(self) -> "TradingEngine":
3         await self._exchange.connect()
4         await self._stream.connect()
5         await self._order.connect()
6         if self._portfolio_manager:
7             await self._portfolio_manager.connect()
8         return self
9
10    async def __aexit__(self, exc_type, exc_val, exc_tb) -> None:
11        await self.stop()
12        await self._stream.disconnect()
13        await self._order.disconnect()
14        await self._exchange.disconnect()
15        if self._portfolio_manager:
16            await self._portfolio_manager.disconnect()

```

El context manager asíncrono permite usar el motor de forma segura en tests:

```

1 async def test_trading_engine():
2     async with TradingEngine(...) as engine:
3         # Todas las conexiones están abiertas
4         await engine.run()
5         # Al salir del bloque, todas se cierran automáticamente

```

Esto garantiza que las conexiones siempre se cierren correctamente, incluso si ocurre una excepción, mejorando la robustez del código y facilitando el testing.

8.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Diseña un decorador que mida latencia de funciones críticas sin alterar su firma.

Veamos las implementaciones de los dos decoradores para medir la latencia, tipados genéricamente usando `ParamSpec` y `TypeVar` para preservar completamente la firma de la función decorada:

```

1 import functools
2 import logging
3 import time
4 from typing import Callable, TypeVar, ParamSpec
5
6 P = ParamSpec("P")
7 R = TypeVar("R")
8
9 logger = logging.getLogger(__name__)
10
11 def timed_async(func: Callable[P, R]) -> Callable[P, R]:

```

```

12     """
13     Decorator that measures the time it takes an async function to
        execute.
14     """
15     @functools.wraps(func)
16     async def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
17         start = time.perf_counter()
18         try:
19             result = await func(*args, **kwargs)
20             return result
21         finally:
22             elapsed_ms = (time.perf_counter() - start) * 1000
23             logger.debug(f"{func.__name__} completed in {elapsed_ms:.2f}
                ms")
24     return wrapper
25
26 def timed(func: Callable[P, R]) -> Callable[P, R]:
27     """
28     Decorator that measures the time it takes a sync function to execute
        .
29     """
30     @functools.wraps(func)
31     def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
32         start = time.perf_counter()
33         try:
34             result = func(*args, **kwargs)
35             return result
36         finally:
37             elapsed_ms = (time.perf_counter() - start) * 1000
38             logger.debug(f"{func.__name__} completed in {elapsed_ms:.2f}
                ms")
39     return wrapper

```

Características clave:

- **Preservación de firma:** ParamSpec y TypeVar permiten que el tipo de retorno y los parámetros se preserven completamente, manteniendo el tipado estático.
- **functools.wraps:** Preserva metadatos como `__name__`, `__doc__` y `__annotations__`.
- **try/finally:** Garantiza que el tiempo siempre se registre, incluso si la función lanza una excepción.
- **time.perf_counter():** Usa un contador de alta precisión para mediciones precisas de latencia.
- **logger.debug():** Registra en nivel debug para no saturar los logs en producción.

Uso en el sistema:

```

1 @timed_async
2 async def buy_market(self, symbol: Symbol, quantity: Quantity) ->
    OrderResponse:
3     """Buy at current market price."""
4     # La firma y el tipado se preservan completamente
5     ...

```

Pregunta Avanzada

Explica cómo una metaclass puede validar automáticamente la implementación de un puerto hexagonal.

Antes hemos visto la implementación de la metaclass `AdapterMeta` que valida automáticamente la implementación de puertos hexagonales mediante el hook `__new__`, que se ejecuta durante la creación de la clase.

El proceso funciona así:

1. **Detección automática del puerto:** Busca en las clases base una que termine en "Port" y sea subclase de `ABC`.
2. **Identificación de métodos abstractos:** Recorre los métodos del puerto y detecta cuáles están marcados con `@abstractmethod`.
3. **Validación de implementación:** Verifica que todos los métodos abstractos estén implementados en el adaptador.
4. **Validación de firmas:** Usa `inspect.signature()` para comparar los parámetros de cada método con los del puerto.
5. **Reporte de errores:** Si hay métodos faltantes o firmas incorrectas, lanza un `TypeError` con un mensaje detallado.

Ejemplo de uso:

```
1 class StoragePort(ABC):
2     @abstractmethod
3     async def save_candle(self, candle): ...
4
5     @abstractmethod
6     async def get_candles(self, symbol, interval): ...
7
8 class PostgreAdapter(StoragePort, metaclass=AdapterMeta):
9     async def save_candle(self, candle):
10         # Implementación correcta
11         ...
12
13     # Error detectado al definir la clase:
14     # TypeError: Adapter 'PostgreAdapter' does not comply with '
15     # StoragePort':
16     # Methods not implemented: ['get_candles']
```

Beneficios de esta validación:

- **Seguridad de tipos en tiempo de definición:** Los errores se detectan antes de instanciar la clase.
- **Cumplimiento del contrato:** Garantiza que los adaptadores respetan la interfaz del puerto.
- **Arquitectura hexagonal robusta:** Previene errores de implementación que romperían la separación entre dominio e infraestructura.
- **Mejor experiencia de desarrollo:** Mensajes de error claros que indican exactamente qué falta o está mal.

9 Escalado Horizontal y Multiactivo

9.1 Preguntas Básicas [1 pt]

Pregunta Básica

Explica qué rompe cuando pasas de 10 a 300 activos en tu sistema actual.

Actualmente mi sistema de ingesta de datos puede tratar con 300 activos o más. Para tener todas las criptomonedas que hay en Binance solo tendría que incluir los nombres de estas en el archivo de trading-data.ts del frontend.

En el caso de querer guardar velas históricas de los 300 activos también se puede, no desde el front directamente, pero se puede usar el servicio de ingesta de datos para hacerlo, de hecho cuando lo implementaba obtuve varios errores de Binance por exceder el límite de requests. Lo solucioné usando un semáforo que solo permitía pasar un número de requests concurrentes. De esta forma podía descargarme 10000 velas de cada símbolo e intervalo en la base de datos.

Lo que se podría romper es si tengo cientos de Portfolios operando a la vez Binance me podría bloquear la IP por exceder el límite de requests.

Pregunta Básica

Indica los límites de CPU, IO y memoria relevantes.

- **CPU:** Alcanzo los límites de la CPU cuando hago un backtest para poder ajustar unos parámetros de la estrategia de cruce de medias. Usando los 16 procesadores me tardó un tiempo considerable para solamente un intervalo de tiempo.
- **IO:** Cuando he intentado hacer la ingesta de datos a través de la REST API, obtuve errores por parte de Binance ya que no se pueden hacer tantas requests. Otro caso donde esto puede pasar es en conexiones WebSocket abiertas al mismo tiempo, el límite es 2048 por IP y 5 por segundo por conexión. En mi caso no he abierto 2048 WS a la vez pero si pido 10 conexiones en 1 segundo por lo que se ejecutan 5 y luego 5.
- **Memoria:** En el caso de querer guardar las velas históricas de los 300 activos, estas me llegaron a ocupar 7GB de memoria.

9.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Diseña una arquitectura con varias réplicas del backend sin duplicar trabajo.

```

1 # Arquitectura con particionado por hash de símbolo
2 #
3 #   Binance WebSocket
4 #       |
5 #   Load Balancer (hash(symbol) % N)
6 #       |
7 #   +-----+-----+-----+
8 #   |       |       |       |
9 # Backend Backend Backend
10 #    0         1         2

```

```

11 #      /      /      /      /
12 # BTC,ETH SOL,XRP ADA,DOGE
13 #      /      /      /      /
14 #      +-----+-----+-----+
15 #              /
16 #      PostgreSQL (compartido)
17
18 # Cada backend procesa solo sus símbolos asignados
19 # Redis Pub/Sub para coordinar órdenes entre backends

```

Pregunta Avanzada

Explica cómo preparar el sistema para miles de activos sin reescribir el dominio.

El dominio está diseñado para ser agnóstico al número de activos:

1. **Value Objects inmutables:** `Symbol`, `Price`, `Candle` no cambian por escala
2. **Puertos genéricos:** `ExchangePort.get_candles(symbol)` funciona igual con 10 o 10,000 símbolos
3. **Estrategias stateless:** Reciben lista de velas, no dependen de estado global

Los cambios necesarios están en infraestructura:

- Sharding de PostgreSQL por símbolo
- Múltiples instancias de backend con particionado
- Cache distribuido (Redis Cluster)

10 Tolerancia a Fallos y Recuperación Determinista

10.1 Preguntas Básicas [1 pt]

Pregunta Básica

¿Tu sistema garantiza at least once, at most once o exactly once? Justificalo.

El sistema implementa garantías mixtas según el componente:

1. Velas (Candles) - At Least Once con protección a nivel DB

El WebSocket Stream usa `tenacity` con `retry` (hasta 5 intentos) para reconexiones, lo que puede generar mensajes duplicados. Sin embargo, la protección se realiza en dos niveles:

- **Base de datos:** Constraint `UNIQUE (symbol, interval, open_time)` previene duplicados a nivel de base de datos mediante `ON CONFLICT DO NOTHING`.
- **CandleBuffer:** Detecta duplicados por timestamp en memoria y los ignora antes de procesar.

Conclusión: At least once - El sistema puede recibir mensajes duplicados del WebSocket, pero la base de datos y el buffer los previenen. Si falla la inserción en DB, el mensaje se pierde (no hay `retry` de inserción).

2. Posiciones - At Most Once

La base de datos tiene constraint `UNIQUE (portfolio_id, trader_id)` que solo permite una posición abierta por trader. Además, el Trading Engine verifica la existencia de posición antes de crear una orden BUY:

```
1 if signal == Signal.BUY:
2     if self._portfolio_manager.has_position(trader.id):
3         logger.info(f"{trader.id} already has position")
4         return None
```

Conclusión: At most once - Si ya existe una posición, no se crea otra. Si falla la creación, no se reintenta automáticamente.

3. Órdenes de Trading - At Most Once

Si una orden falla, se registra el error pero **NO** se reintenta:

```
1 except Exception as e:
2     logger.error(f"Order failed for {order.trader_id}: {e}")
```

No hay idempotency keys ni verificación de "orden ya ejecutada" antes de enviar. Si se envía la misma orden dos veces, se ejecutará dos veces en Binance.

4. Trades (Operaciones Cerradas) - At Least Once

No hay constraints de unicidad en la tabla `trades`, no hay idempotency keys, y no hay verificación de "trade ya registrado" antes de insertar. Pueden existir trades duplicados si se ejecuta la misma orden múltiples veces o si hay fallos en el sistema.

Pregunta Básica

Define qué ocurre si la persistencia falla temporalmente.

Si PostgreSQL falla en medio de una operación, el pool de `asyncpg` usa transacciones `transaction()` que garantizan que, si ocurre algún error, se hace *rollback* completo y ningún cambio parcial llega a la base de datos. Una vez la base de datos se recupera, puedo reintentar la operación sabiendo que el estado anterior sigue siendo consistente.

11 Pruebas Avanzadas y Validación del Sistema

11.1 Preguntas Básicas [1 pt]

Pregunta Básica

Explica cómo escribiste tests unitarios para tus estrategias.

He implementado tests unitarios para las estrategias `mean_cross` y `momentum` usando `pytest`. Los tests verifican la generación correcta de señales (BUY, SELL, HOLD) bajo diferentes condiciones de mercado.

Funciones auxiliares (helpers)

Para evitar repetir código, definí funciones auxiliares que crean velas (`Candle_static`) de forma sencilla:

- `create_candle(close_price, index)`: Genera una vela con open/high/low/close/volume/timestamp coherentes a partir de un precio de cierre.
- `create_candles_with_prices(prices)`: Construye una lista de velas a partir de una lista de precios de cierre.

Tests para Estrategia de Cruce de Medias (`test_estrategia_medias.py`)

1. **Inicialización**: Verifico que los valores por defecto y personalizados de `slow_period`, `fast_period` y `min_candles_required` se calculan correctamente.
2. **Casos de HOLD**:
 - Cuando no hay suficientes velas (`len(candles) < slow_period + 1`) $\rightarrow$ `Signal.HOLD`
 - Cuando no hay cruce (precios planos o tendencia sin cambio de relación entre medias) $\rightarrow$ `Signal.HOLD`
 - Sin nuevo cruce: casos donde la rápida ya estaba por encima/por debajo de la lenta y sigue igual $\rightarrow$ `Signal.HOLD` (para comprobar que solo se dispara en el momento del cruce)
3. **Golden Cross (BUY)**: Construyo una serie donde la media rápida cruza al alza la media lenta (precios bajando y luego subiendo fuerte) y espero `Signal.BUY`.
4. **Death Cross (SELL)**: Serie donde la media rápida cruza a la baja la lenta (precios suben y luego caen fuerte) y espero `Signal.SELL`.
5. **Nombre de la estrategia**: Compruebo que el `__name__` se formatea como `"MA Cross (fast/slow)"`.

Tests para Estrategia de Momentum (`test_momentum.py`)

1. **HOLD por pocas velas**: Menos velas que `reference_period + 1` $\rightarrow$ `Signal.HOLD`
2. **BUY por momentum alcista**: Serie donde el precio sube desde 100 a 105 ($\approx +5\%$) con `threshold = 0.02` $\rightarrow$ `Signal.BUY`
3. **SELL por momentum bajista**: Serie donde baja de 100 a 95 ($\approx -5\%$) con `threshold = 0.02` $\rightarrow$ `Signal.SELL`
4. **HOLD dentro del umbral**: Cambio pequeño ($\approx +1\%$ con `threshold = 0.05`) $\rightarrow$ `Signal.HOLD`

Pregunta Básica

¿Qué es un fake exchange y para qué sirve en testing?

Un **fake exchange** es una implementación del puerto `ExchangePort` que simula el comportamiento del exchange real sin hacer llamadas de red. Yo tengo mocks de todos mis adaptadores para utilizarlos en los tests sin tener que conectarme realmente a Binance o a la base de datos. Por ejemplo:

```

1
2 class MockStreamAdapter(StreamPort, metaclass=AdapterMeta):
3     """Mock stream adapter"""
4     def __init__(self, base_price: float = 100.0, delay: float =
5         1.0) -> None:
6         self._base_price = base_price
7         self._delay = delay
8         self._running = False
9
10    async def get_live_candle(self, symbol: Symbol, interval:
11        Interval) -> Candle:
12        pass
13
14    async def stream_candle(self, symbol: Symbol, interval: Interval
15        ) -> AsyncIterator[Candle]:
16        """Stream a candle from the mock stream"""
17        self._running = True
18
19        while self._running:
20            price = self._base_price + random.uniform(-5, 5)
21
22            candle = Candle.from_static(Candle_static(
23                symbol=symbol,
24                interval=interval,
25                open=Price(Decimal(str(price))),
26                high=Price(Decimal(str(price + 1))),
27                low=Price(Decimal(str(price - 1))),
28                close=Price(Decimal(str(price + 0.5))),
29                volume=Quantity(Decimal("100")),
30                timestamp=Timestamp(datetime.now()))
31            ))
32            candle.is_closed = True # Simular vela cerrada
33
34            yield candle
35            await asyncio.sleep(self._delay)
36
37    async def connect(self) -> "StreamPort":
38        """Connect to the stream."""
39        self._running = True
40        return self
41
42    async def disconnect(self) -> None:
43        """Disconnect from the stream."""
44        self._running = False

```

Sirve para:

- Tests rápidos sin latencia de red
- Tests deterministas (mismo input = mismo output)
- Simular escenarios difíciles (errores, timeouts)
- No consumir rate limit del exchange real

11.2 Preguntas Avanzadas [1 pt]

Pregunta Avanzada

Explica cómo simular condiciones de red adversas.

He intentado conseguir que el WebSocket de Binance me diese dos velas desordenadas he simulado que se cae la conexión cada cierto tiempo y luego te vuelves a conectar. Primero establezco la conexión y pido velas y a los pocos segundos rompo el bucle y la conexión, a los pocos segundos me vuelvo a conectar. No conseguí velas desordenadas pero sí observé que a veces el ingestion time era menor al event time de la vela.

```
1  async def test_ws_desorden(ws: BinanceStreamAdapter) -> None:
2
3
4  velas: List[Candle] = []
5
6  for i in range(REPEATS):
7
8      logger.info(f"Test {i+1} of {REPEATS}")
9
10
11
12     await ws.connect()
13     logger.info(f"Connected to {SYMBOL} {INTERVAL}")
14     start_time = time.time()
15     try:
16
17         async for candle in ws.stream_candle(SYMBOL, INTERVAL):
18             print(candle)
19
20             velas.append(candle)
21
22             if time.time() - start_time > TIME_CONNECTED:
23                 logger.info(f"Time connected to {SYMBOL} {INTERVAL}
24                             exceeded {TIME_CONNECTED} seconds")
25                 break
26
27     except Exception as e:
28         logger.error(f"Error in test {i+1}: {e}")
29         continue
30
31     await ws.disconnect()
32
33
34     logger.info(f"Down for {TIME_DOWN} seconds")
35     await asyncio.sleep(TIME_DOWN)
```

Pregunta Avanzada

Describe cómo verificar que tu pipeline es reproducible: mismo input, mismo estado final.

Puedes usar los mocks mencionados antes, en los mocks puedes crear casos totalmente deterministas que siempre den el mismo output usando el mismo seed.

En mi caso he creado un mock de ExchangePort que genera velas de forma aleatoria pero con un seed fijo para que siempre genere las mismas velas. Solo necesitas añadir un método de clase de esta forma:

```

1  class MockExchangeAdapter(ExchangePort, metaclass=AdapterMeta):
2      """
3      Mock exchange adapter for testing.
4      Generates deterministic candle data with correct chronological order
5      """
6
7      def __init__(self, base_price: float = 100.0, seed: Optional[int] =
      None):
8          self._base_price = base_price
9          # RNG instanciado para reproducibilidad
10         self._rng = random.Random(seed) if seed is not None else random.
            Random()
11
12     async def connect(self) -> None:
13         """Connect to the mock exchange (no-op)"""
14         pass
15
16     async def disconnect(self) -> None:
17         """Disconnect from the mock exchange (no-op)"""
18         pass
19
20     @classmethod
21     def deterministic(cls, base_price: float = 100.0, seed: int = 101) ->
        MockExchangeAdapter:
22         return cls(base_price, seed)

```

12 Arquitectura del Sistema

12.1 Arquitectura Hexagonal

El sistema implementa arquitectura hexagonal con separación clara entre:

- **Dominio:** Value objects (`Price`, `Symbol`), entities (`Candle`, `Trade`), y puertos (interfaces abstractas). No depende de ninguna librería externa.
- **Aplicación:** Servicios que orquestan el flujo (`DataIngestionService`, `BacktestService`). Usan puertos, no implementaciones.
- **Infraestructura:** Adaptadores que implementan los puertos (`BinanceAdapter`, `PostgresAdapter`). Aquí viven las dependencias de librerías externas (`aihttp`, `asyncpg`).

13 Conclusiones

13.1 Retos Encontrados

1. **Rate limiting de Binance:** Resuelto con `asyncio.Semaphore` limitando a 10 requests concurrentes.

2. **Reconexión de WebSocket:** Implementé exponential backoff con tenacity para manejar desconexiones.
3. **FastAPI:** Familiarizarse con la librería FastAPI y su documentación.
4. **Testing:** Aprender los básicos de pytest.

13.2 Decisiones de Simplificación

1. **Solo estrategias long:** No implementé shorts por simplicidad.
2. **Sin órdenes reales:** El sistema genera señales y opera en una paper account.
3. **Single-node:** No implementé sharding de base de datos.

13.3 Posibles Mejoras Futuras

1. Implementar CI/CD con GitHub Actions
2. Desplegar en AWS con Terraform
3. Añadir Redis para caching
4. Añadir más estrategias (RSI, MACD, Bollinger Bands)
5. Implementar backtesting con slippage y comisiones
6. Mejorar la concurrencia y eliminar race conditions

13.4 Comentarios

1. No he podido incluir las estrategias basadas en patrones de velas, he tenido bastantes bugs a última hora y he decidido quitarlas para poder entregar el proyecto.
2. Al dockerizar la aplicación me han aparecido múltiples bugs de última hora que he tenido que arreglar, hay algunos que no he podido resolver y he tenido que quitarlos: las etiquetas que te permiten ver en una gráfica cuando se ha comprado y vendido un activo, salían en la vela equivocada y las he quitado, puedes ver los trades debajo de la gráfica. (En el backtest sí aparecen los trades correctamente).
3. Según he empezado a hacer la memoria se me ocurrían cosas para cambiar, pero no he podido implementarlas a última hora.

Referencias

- [1] Binance. *Binance API Documentation*. 2024. URL: <https://binance-docs.github.io/apidocs/spot/en/> (visited on 01/15/2026).
- [2] Binance. *Binance WebSocket Streams*. 2024. URL: <https://binance-docs.github.io/apidocs/spot/en/#websocket-market-streams> (visited on 01/15/2026).
- [3] Julien Danjou. *Tenacity Documentation*. 2024. URL: <https://tenacity.readthedocs.io/> (visited on 01/15/2026).
- [4] Docker, Inc. *Docker Documentation*. 2024. URL: <https://docs.docker.com/> (visited on 01/15/2026).
- [5] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN: 978-0201633610.
- [6] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN: 978-0134494166.
- [7] Meta Platforms, Inc. *React Documentation*. 2024. URL: <https://react.dev/> (visited on 01/15/2026).
- [8] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/> (visited on 01/15/2026).
- [9] Python Software Foundation. *asyncio — Asynchronous I/O*. 2024. URL: <https://docs.python.org/3/library/asyncio.html> (visited on 01/15/2026).
- [10] Sebastián Ramírez. *FastAPI Documentation*. 2024. URL: <https://fastapi.tiangolo.com/> (visited on 01/15/2026).
- [11] Evan You. *Vite Documentation*. 2024. URL: <https://vitejs.dev/> (visited on 01/15/2026).